PRACTICAL - 04

Aim: Password strength (PasswordChecker.java + Driver.java)

Code Screenshot:

PasswordChecker.java

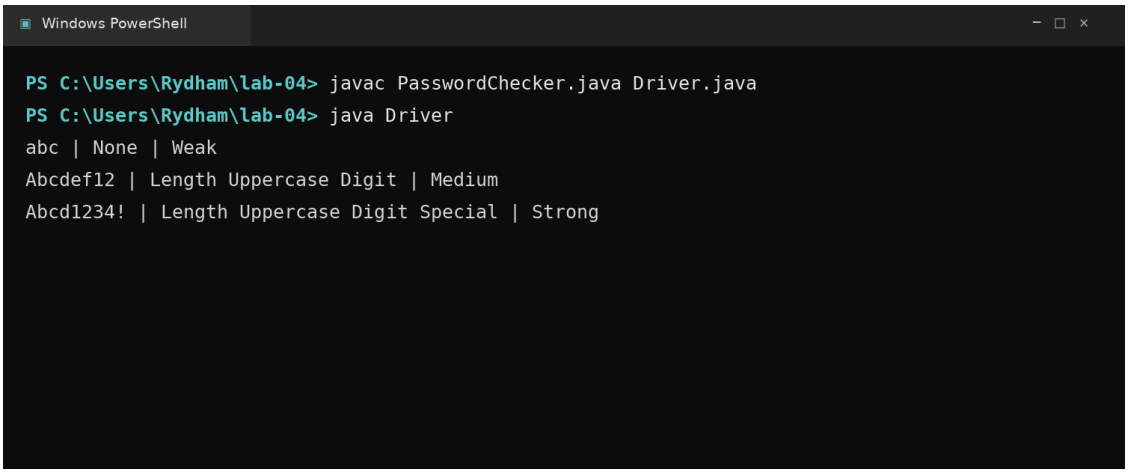
```java
public class PasswordChecker {
    public String strength(String pw) {
        int score = 0;
        if (pw.length() >= 8) score++;
        if (pw.matches(".*[A-Z].*")) score++;
        if (pw.matches(".*[0-9].*")) score++;
        if (pw.matches(".*[^A-Za-z0-9].*")) score++;
        if (score <= 1) return "Weak";
        if (score <= 3) return "Medium";
        return "Strong";
    }

    public void passedRules(String pw) {
        boolean found = false;
        if (pw.length() >= 8) { System.out.print("Length "); found = true; }
        if (pw.matches(".*[A-Z].*")) { System.out.print("Uppercase "); found = true; }
        if (pw.matches(".*[0-9].*")) { System.out.print("Digit "); found = true; }
        if (pw.matches(".*[^A-Za-z0-9].*")) { System.out.print("Special "); found = true; }
        if (!found) System.out.print("None");
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        PasswordChecker checker = new PasswordChecker();
        String[] passwords = {"abc", "Abcdef12", "Abcd1234!"};
        for (String pw : passwords) {
            System.out.print(pw + " | ");
            checker.passedRules(pw);
            System.out.println(" | " + checker.strength(pw));
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned to validate passwords using multiple security rules.
- Understood the use of Regular Expressions (Regex) for pattern matching.
- Learned to use String methods such as length() and matches().
- Implemented password strength classification (Weak, Medium, Strong).
- Improved problem-solving and decision-making using conditional statements.

PRACTICAL - 04

Aim: Chat-log filter (ChatFilter.java + Driver.java)

Code Screenshot:

ChatFilter.java

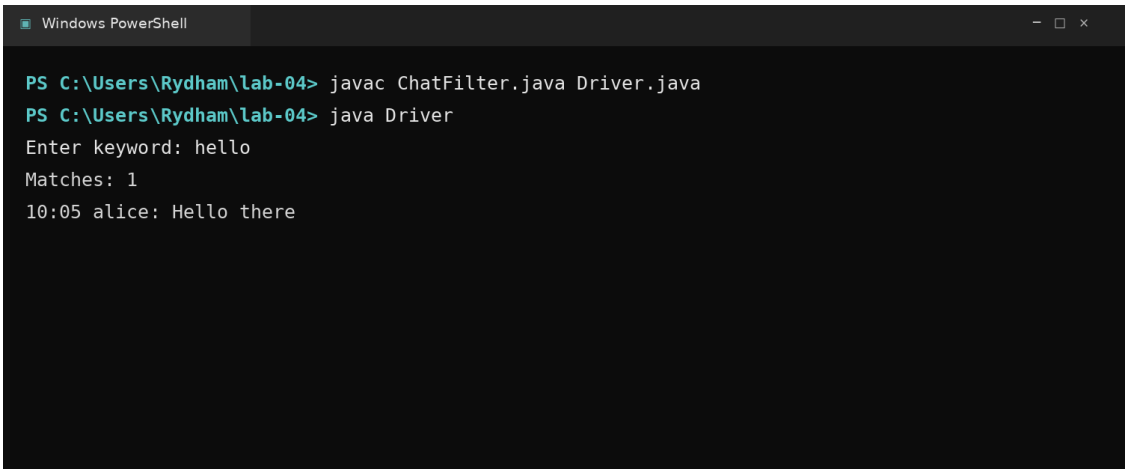
```java
public class ChatFilter {
    public String filter(String[] lines, String keyword) {
        StringBuilder report = new StringBuilder();
        int count = 0;
        for (String line : lines) {
            String[] parts = line.split(" ", 3);
            if (parts.length < 3) continue;
            if (parts[2].toLowerCase().contains(keyword.toLowerCase())) {
                count++;
                report.append(parts[0]).append(" ").append(parts[1])
                      .append(": ").append(parts[2]).append("\n");
            }
        }
        return "Matches: " + count + "\n" + report;
    }
}
```

Driver.java

```java
import java.util.Scanner;

public class Driver {
    public static void main(String[] args) {
        String[] logs = {
            "10:05 alice Hello there",
            "10:06 bob Meeting at 5",
            "10:07"
        };
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter keyword: ");
        String keyword = sc.nextLine();
        ChatFilter filter = new ChatFilter();
        System.out.print(filter.filter(logs, keyword));
        sc.close();
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned to process and manipulate strings using split().
- Understood the use of StringBuilder for efficient string concatenation.
- Learned case-insensitive searching using toLowerCase() and contains().
- Implemented filtering of chat messages based on user input.
- Improved understanding of arrays, loops, and string processing techniques.

PRACTICAL - 04

Aim: Template filler (TemplateFiller.java + Driver.java)

Code Screenshot:

TemplateFiller.java

```java
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class TemplateFiller {
    public String fill(String template, String[] names, String[] values) {
        Pattern pattern = Pattern.compile("\\{(\\w+)\\}");
        Matcher matcher = pattern.matcher(template);
        StringBuilder result = new StringBuilder();
        int last = 0;
        while (matcher.find()) {
            result.append(template, last, matcher.start());
            String key = matcher.group(1);
            String value = "[?]";
            for (int i = 0; i < names.length; i++)
                if (names[i].equals(key)) { value = values[i]; break; }
            result.append(value);
            last = matcher.end();
        }
        result.append(template.substring(last));
        return result.toString();
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        String template = "Dear {name}, order {id} ships {date}.";
        String[] names = {"name", "id"};
        String[] values = {"Riya", "A07"};
        TemplateFiller filler = new TemplateFiller();
        System.out.println(filler.fill(template, names, values));
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned to use Pattern and Matcher for regular expression processing.
- Extracted placeholder names using capturing groups.
- Used StringBuilder to construct the filled template.
- Replaced known placeholders using parallel arrays.
- Handled missing values with the default [?] marker.